

ROTONDO FARMS — NPU-POWERED PLANT HEALTH NARRATION

Arm Create: AI Optimization Challenge — Physical AI Track

Draft submission text — fill in / trim as needed before pasting into Devpost

LINKS

Repo: <https://github.com/Nicholas-Rotondo/rotondo-farms-npu>

Demo video: <https://youtu.be/mhgFhElPhOE>

PROJECT DESCRIPTION (short, top-of-page summary)

An automated indoor grow system that reads live soil moisture from real sensors and uses a locally-run LLM (Qwen3-0.6B) on the Orange Pi 5's NPU to generate natural-language plant health narrations — with a measured, side-by-side CPU vs. NPU benchmark showing where on-device NPU inference actually pays off, and where it doesn't.

PROJECT OVERVIEW

Rotondo Farms is a physical, working grow-cabinet monitoring system. Two capacitive soil moisture sensors (tomato and pepper) feed into an ADS1115 ADC connected to an Orange Pi 5 (RK3588S) over I2C. Instead of just displaying raw numbers on a dashboard, the system runs a small local language model on the Orange Pi's NPU to turn those readings into a plain-English summary of plant health — the kind of thing a person could glance instead of interpreting a moisture percentage.

The core engineering question this project set out to answer for Track 1 (Optimization) was concrete: for a small, real-time inference workload like this, is the NPU actually faster than the CPU on this hardware — and if no, why not? That required getting a model running natively on the NPU at all (not a given — see "Technical Challenges" below), then running a controlled, multi-trial benchmark comparing the two paths on identical input.

FUNCTIONALITY / OUTPUT SUMMARY

- Reads live raw ADC values from 2 capacitive soil moisture sensors via ADS1115 (I2C), on an Orange Pi 5.
- Classifies each reading as dry / ok / wet against calibrated thresholds.
- Builds a prompt from the live readings and sends it to a locally-hosted RKLLama server running Qwen3-0.6B, either on CPU (GGUF format) or on the RK3588S NPU (native .rkllm format).
- Returns a short, natural-language narration of overall plant health, e.g.: "The sensor readings indicate that the Tomato is at a dry state ... while the Pepper is healthy... requires attention to maintain soil health."
- Logs every inference call's timing and token counts to a CSV for benchmarking.
- Supports a --trials mode that runs N back-to-back generations against a fixed prompt for a controlled CPU-vs-NPU comparison.

BENCHMARK RESULT (8 trials each, same prompt, same hardware):

CPU (qwen3-0.6b, GGUF format): avg 39.15 tok/s (range 25.9-41.7)

NPU (qwen3-0.6b-npu, native .rkllm): avg 13.43 tok/s (range 12.4-14.4)

[INSERT cpu_vs_npu_benchmark.png HERE]

Counter to the initial assumption, CPU outperformed the NPU for this specific model size. The most likely explanation: at 0.6B parameters, the model is small enough that the NPU's per-request dispatch and initialization overhead outweighs its parallel-processing advantage. NPU acceleration tends to show its real benefit on larger models or batched workloads, not single small requests like this one. This is itself a useful, honest optimization finding — it tells a real deployer of this kind of system when NPU offload is and isn't worth the added complexity.

TECHNICAL CHALLENGES / WHAT WE OPTIMIZED

Getting a model to genuinely run on the NPU (not silently fall back to CPU) required real debugging work, not just following a tutorial:

1. The first working NPU path used a GGUF-format model, which turned out to be running on CPU the entire time despite appearing to work (/sys/kernel/debug/rknpu/load stayed at 0%) — GGUF-on-NPU support in the inference server used (RKLLama) is explicitly experimental.
2. Switched to the natively-supported model format (.rkllm) via

pre-converted Hugging Face models — this got real NPU utilization (confirmed 3-50% load), but generation was truncating mid-sentence consistently across three different model files and sizes.

3. Root-caused the truncation to an ABI mismatch: the RKLLM runtime library (librkllmrt.so) had been updated to a new version with a different C struct layout, but a known-good older runtime version was needed for stable generation on this hardware — and pairing an old runtime library with new application code (mismatched struct field offsets) silently corrupted a parameter passed into the native library. Fixed by pinning both the runtime library and the application code to the last mutually compatible commit.
4. Also diagnosed and fixed: a background systemd service silently competing for port 8080 with manually-started test instances, and a pipeline bug where the API wasn't explicitly told to switch models between requests.

SETUP / BUILD / RUN INSTRUCTIONS (Arm64 — Orange Pi 5, RK3588S)

Hardware required:

- Orange Pi 5 (RK3588S), Armbian, kernel 6.1.115-vendor-rk35xx
- ADS1115 ADC breakout, address 0x48, wired to i2c-5 (header pins 3/5)
- 1-4 capacitive soil moisture sensors on ADS1115 channels A0-A3

Software setup:

1. Clone the repo:
git clone https://github.com/Nicholas-Rotondo/rotondo-farms-npu
2. Set up RKLLama (NPU inference server) per its own install docs:
<https://github.com/NotPunchnox/rkllama>
3. Create a Python venv and install dependencies:
python3 -m venv rkllama-env
source rkllama-env/bin/activate
pip install requests smbus2
4. Pull the NPU model (native .rkllm format):
rkllama_client pull
Repo ID: dulimov/Qwen3-0.6B-rk3588-1.2.1-unslloth
File: Qwen3-0.6B-rk3588-w8a8-opt-1-hybrid-ratio-0.0.rkllm
Custom Model Name: qwen3-0.6b-npu
5. Start the RKLLama server:
rkllama_server --models ~/models --llamacpp <path-to-llama.cpp-build>/bin

Running the pipeline:

```
python3 scripts/sensor_narration.py
```

(single run: reads live sensors, prints narration)

```
python3 scripts/sensor_narration.py --loop 14400
```

(continuous mode, runs every 4 hours)

```
python3 scripts/sensor_narration.py --model qwen3-0.6b --trials 8
```

```
python3 scripts/sensor_narration.py --model qwen3-0.6b-npu --trials 8
```

(benchmark mode: N trials against a fixed prompt, for CPU vs NPU comparison; logs to npu_benchmark.csv)

Validating it worked:

- Console output shows live sensor readings per channel, followed by a full natural-language narration.
- npu_benchmark.csv accumulates one row per inference call with timestamp, model, token counts, elapsed time, and tokens/sec.
- NPU utilization can be independently confirmed via:
`cat /sys/kernel/debug/rknpu/load`